Sparse Autoencoders (SAE) for LLM Interpretability

From superposition to monosemantic features

ML 330 Project

ANTLR - Nafis Nahian, Arnob Biswas, Tanvir Liaquat Uday

Journey to SAE-based Model Interpretation

1. Understanding the superposition problem in LLMs
2. Collecting intuition on how SAE can help
3. SAE technical framework from recent survey literature
4. Architectural shift from vanilla SAE to TopK/Batch-TopK/FC variants
5. Training workflow and health checks
6. Evaluation: reconstruction, sparsity, and feature behavior
7. Interpretation of learned feature families and limitations
8. Inference UI walkthrough for practical feature analysis

Superposition in LLMs

Core issue

- LLM activations can encode more independent features than available neurons
- A single neuron may respond to multiple unrelated concepts
- This appears as **polysemanticity**, a signature of superposition

Consequence

- Neuron-level interpretation becomes unstable and ambiguous
- Hard to assign one clean concept to one neuron

Superposition Intuition

Geometric view

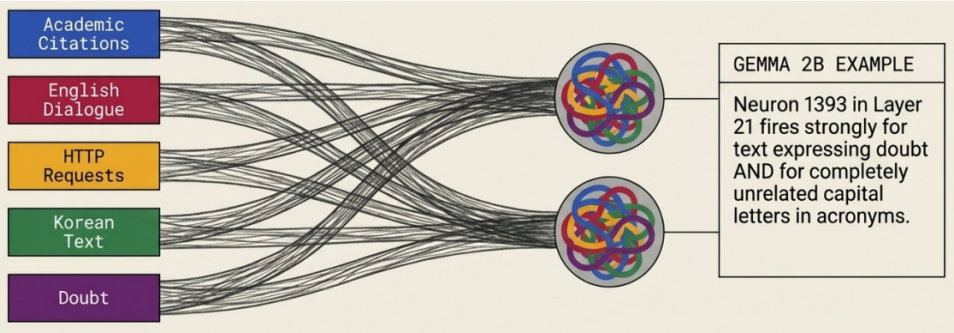
- Latent space has dimension d
- Model needs to represent m concepts with $m \gg d$
- Concepts are packed as overlapping linear combinations

Observed activation vector:

$$z \approx \sum_{i=1}^{m} a_i f_i$$

- f_i: feature directions
- a_i: feature strengths

Superposition Visual Intuition



Why This Hurts Interpretability

- Raw neuron activations are not aligned with human concepts
- A neuron can fire for different causes in different contexts
- Causal interventions at neuron level can be noisy

Goal of SAE-based Interpretibility

- Decompose dense/polysemantic representations
- Recover sparse, more monosemantic feature activations

How Sparsity Solves Superposition

Key idea: sparse dictionary coding

Learn:

- an overcomplete dictionary ($m > d$)
- sparse coefficients so only a few features are active per token

Then each input is reconstructed by a small active subset:

$$z \to h(z) \to \hat{z}$$

- Sparsity encourages feature specialization
- Specialization improves interpretability and causal tracing

Sparse Representation Principle

Given encoder output $h(z)$:

- Most entries in $h(z)$ should be 0 (or near 0)
- Only informative features should activate

This pushes the model away from:

- "everything somewhat active"

Toward:

- "few meaningful features strongly active"

Result: cleaner feature-to-concept mapping.

SAE Intro: Architecture

Standard SAE components:

1. **Encoder**: $z \in \mathbb{R}^d \to h(z) \in \mathbb{R}^m$
2. **Decoder**: reconstruct $\hat{z}$ from sparse code

Typical form:

$$\hat{z} = h(z)W_{dec} + b_{dec}$$

- Overcomplete regime: $m \gg d$
- Sparse features are easier to interpret

	LLM Residual Stream (Dense)	SAE Output (Sparse)
Architecture	Compressed, fewer neurons than features.	Overcomplete, vastly more dictionary elements than inputs.
Encoding	Superposition / Polysemantic.	Sparse / Monosemantic.
Compute	Highly computationally efficient.	Computationally expensive.
Interpretability	Uninterpretable (Black Box).	Highly interpretable.

SAE Loss (L1-Regularized Form)

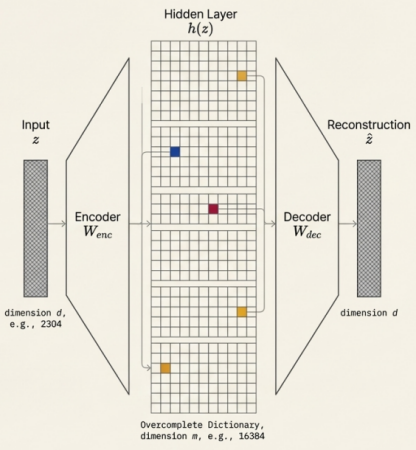
From the survey's standard objective:

$$L(z) = \|z - \hat{z}\|_2^2 + \alpha \|h(z)\|_1$$

- Reconstruction term keeps information fidelity
- L_1 term enforces sparse activations
- α controls the trade-off

Trade-off

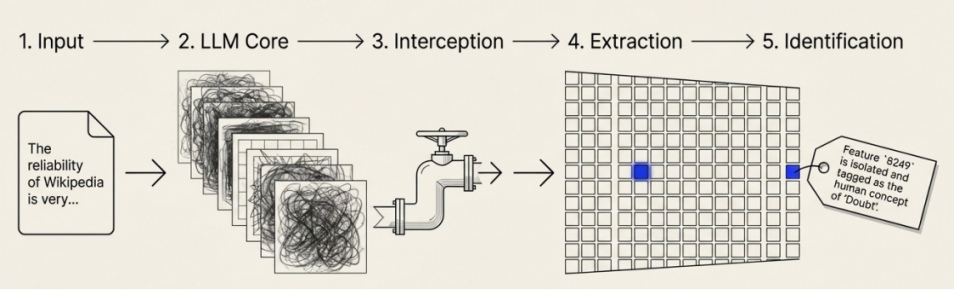
- High α: more sparse, weaker reconstruction
- Low α: better reconstruction, less disentanglement



Research Setting

Architecture	Dataset	GPT-2 Layer
Vanilla SAE	BookCorpus	Layer 8
TopK SAE	Openwebtext	Layer 9
Feature Choice SAE	MLCommons/peoples_speech	Layer 10

Project Workflow



- Sample layer activations z from a target LLM layer
- Train SAE with reconstruction + L1 sparsity objective
- Iterate with diagnostics: dead units, density, reconstruction quality

Architectural Comparison (Quick)

Variant	Sparsity mechanism	Strength	Main concern
Vanilla SAE	L_1 on activations	Simple and standard	Shrinkage bias
TopK SAE	Hard top-K per token	Direct control, cleaner sparsity	Choosing good K
FC_SAE	Token allocation per feature	Better feature usage	Assignment complexity

Common Failure Cases

- Dead features: never selected under current data and hyperparameters
- Split concepts: one concept fragmented across many weak features
- Polysemantic leftovers: one feature still mixes unrelated contexts
- Dataset bias: labels may reflect corpus skew

Tuning Parameters (Practical)

Core knobs

- Dictionary size m (expansion factor)
- Sparsity strength: α (L1 regularization)
- Learning rate and optimizer settings
- Batch size and token sampling strategy

Typical tuning workflow

1. Start with stable reconstruction
2. Increase sparsity gradually
3. Track dead features and usage spread
4. Select checkpoint with best interpretability/reconstruction balance

From Survey Taxonomy to Practice

The survey groups SAE evolution into two broad directions:

- **Architecture improvements**: Gated, TopK, JumpReLU, Feature Choice, etc.
- **Training/evaluation improvements**: better sparsity control, dead-feature mitigation, stronger diagnostics

Practical takeaway
Do not optimize only reconstruction loss; monitor sparsity quality and feature utilization jointly.

TopK SAE and Batch TopK Intuition

TopK SAE enforces sparsity by selection, not by L_1 penalty.

Encoder idea:

$$\tilde{h}(z) = TopK\left(W_{enc}(z - b_{pre})\right)$$

Training objective (survey form):

$$L(z) = \|z - \hat{z}\|_2^2 + \alpha L_{aux}$$

- Keep only top-K activations per token
- Avoid direct magnitude shrinkage from L1
- Batch TopK stabilizes activation budget across a batch

FC_SAE (Feature Choice SAE)

Feature Choice SAE changes sparsity allocation direction.

Instead of:

- fixed active features per token

It enforces:

- each feature gets assigned to a controlled number of tokens

Survey statement (Ayonrinde, 2024):

$$\sum_j S_{i,j} = m, \ \forall i$$

- $S_{i,j}$ indicates whether feature i is active for token j
- Promotes more uniform feature utilization
- Targets dead-feature collapse directly

Dataset Ablation: Vanilla SAE

Metric	peoples_speech	openwebtext
Feature density	1.20%	1.26%
Dead features	4716	10177
Dead feature fraction	38.38%	55.21%

Interpretation:

- openwebtext shows much higher dead-feature collapse than peoples_speech (55.21% vs 38.38%).
- A likely reason is dataset composition: openwebtext is dominated by news/political style text, so feature usage is less diverse.
- peoples_speech has broader spoken-language variability, activating a wider portion of the dictionary.

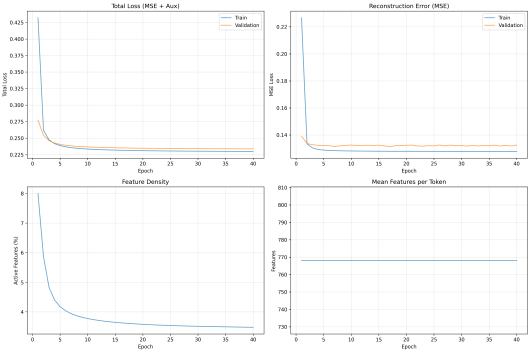
Architectural Shift: Why Move Beyond Vanilla SAE?

Metric	Vanilla SAE	FC_SAE
MSE (reconstruction)	0.1264	0.1597
Cosine similarity	0.9368	0.9213
Explained variance	0.8736	0.8421
Feature density	1.20%	3.48%
Dead features	4716 (38.38%)	13 (0.21%)
Validation loss	0.2575	0.2339

Why we moved to FC_SAE (per survey motivation):

- Vanilla SAE gives stronger raw reconstruction, but suffers severe dead-feature collapse (38.38%).
- FC_SAE greatly improves feature utilization by enforcing allocation pressure across features (rather than only per-token sparsity).
- This supports more reliable interpretability: more features participate, fewer latents are wasted, and coverage is broader across token contexts.
- We accept a modest reconstruction trade-off to gain a much healthier and more usable feature dictionary for analysis.

Full Training Curve Snapshot



- Early training reduces reconstruction error quickly
- Later epochs refine feature specialization
- Best zone: stable reconstruction with healthy utilization

Results: Reconstruction/Interpretability Balance

- Too little sparsity: strong reconstruction, weaker disentanglement
- Too much sparsity: cleaner features, higher information loss
- Practical target: lowest error with stable semantic features

Interpretation Protocol (Survey + Ng Notes)

- Collect top-activating tokens/sentences per feature
- Assign tentative semantic labels from activation evidence
- Validate with counterexamples and cross-context checks
- Flag brittle/polysemantic features for retuning

Ng-style intuition
Interpretable hidden units correspond to sparse non-linear features of input space.

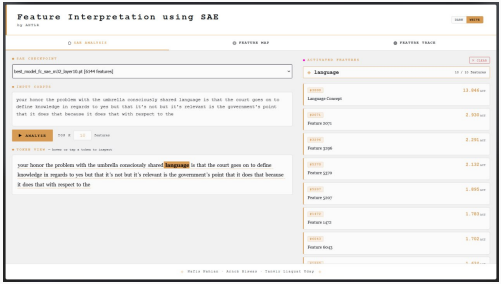
Inference UI Overview

The inference UI (by ANTLR) has **three main tabs**:

Tab	Purpose
SAE Analysis	Input text -> view top-K activated features per token
Feature Map	Search and label features over the dataset
Feature Trace	Trace one feature ID over any input text

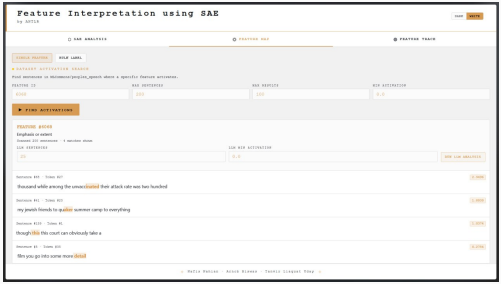
Built on GPT-2 SAE checkpoints (e.g. `best_model_fc_sae_m32_layer10.pt`)

SAE Analysis: Token-level Inspection



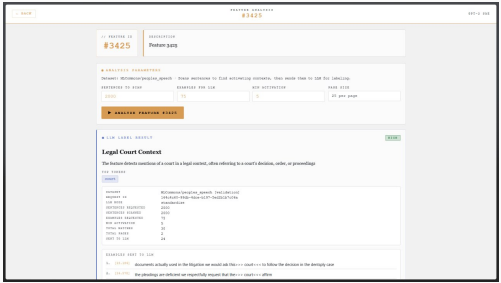
- Paste text and click **Analyze**
- Select a token to view top-K active features

Feature Map: Dataset Activation Search



- Enter a feature ID to scan activation matches
- View matched sentence and exact trigger token

Feature Analysis: LLM Auto-labeling

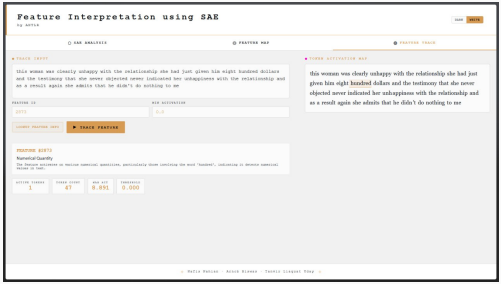


- Sends top activation examples to an LLM for labeling
- Returns semantic label and confidence

Feature Analysis: Activation Examples

- Sentences use »>token«< to mark the trigger token
- Scores support manual validation of labels

Feature Trace: Single-feature Heatmap



- Input text + feature ID highlights activating tokens
- Heat intensity reflects activation strength

Closing

- Superposition motivates feature-level decomposition
- Sparsity makes decomposition practical and interpretable
- Modern SAE variants improve robustness over vanilla L1 baselines
- Reliable interpretation needs architecture and evaluation discipline

References

- Ayonrinde et al. (2024). *Sparse Autoencoders for LLM Interpretability: A Survey*.
- Ng, A. (UFLDL notes). *Sparse Autoencoder*.
- Elhage et al. (2022). *Toy Models of Superposition*.

Thank you.